Name: NGO TUNG CHUONG

ID: s225586848

YouTube: Recorded Video
GitHub: https://github.com/NoXxChapter/SIT221-DataStructure.git

- **Part A**

Miss Marple is at a train station in a foreign town. She wants to select a hotel that has the *maximum number of shortest paths* from the train station. She thinks that this should reduce the risk of getting lost. Suppose she gives you a city map represented via a graph $G = \langle V, E \rangle$ with $n = |V|$ locations and $m = |E|$ edges connecting the locations. Each edge connecting a pair of directly connected locations has a *unit distance*, say 1. Help Miss Marple to find a proper hotel by designing a $O(n + m)$ runtime algorithm that finds the number of shortest paths between the train station, located at node s and every hotel on the map. You need to convince her that your algorithm is correct, and it does find all possible shortest paths. Your solution must contain an explanation and a pseudocode.

Input: Graph G with s as starting node
Output: count[] array where count[v] is the number of shortest paths to v

Pseudocode of BreadthFirstSearch(G, s):

```
Initialize dist[n] to infinity, count[n] to 0, visited[n] to false
dist[s] = 0
count[s] = 1
Create an empty queue Q
Q.enqueue(s) // put start node in queue
visited[s] = true // we set start node as visited

// we utilize the First In-First Out rule
while Q is not empty:
    currentNode = Q.dequeue()
    for each neighbor i of currentNode:
        if not visited[i]:
            visited[i] = true
            dist[i] = dist[currentNode] + 1
            count[i] = 1 // we found first shortest path to reach this node
            Q.enqueue(i)
        else if dist[i] == dist[currentNode] + 1:
            count[i] = count[i] + 1 // accumulate the shortest paths that we found
return count
```

Breadth First Search explores nodes **layer by layer**. So, when we **first encounter** a neighbor node i from currentNode, we know right away that is the shortest distance. But if any other node that reaches that neighbor node we **already visited** with that **same distance**. We can I **accumulate another alternative shortest path**, by that we can **know all the shortest paths** we can travel to reach that desire node.

- **Part B**

A communication network, such as the Internet, can be modelled as an undirected graph $G = \langle V, E \rangle$. Here, the vertices of set V are the computers of the network, and the edge set E consists of one edge for each pair of computers that are directly connected. We assume that the edges of G are undirected; that is, if there is a direct connection from computer u to computer v, then there is also a direct connection from computer v to computer u.

It is highly desirable for a communication network graph to be ***connected*** so that every computer on the network can communicate, possibly through a series of relays, with any other computer. But networks can change, with some computers failing and other computers being added to the network. It is useful to have a *testing algorithm* that collects information about the current network graph (vertices and edges) at designated times and determines properties related to connectivity.

Describe, in words and a pseudocode, a ***testing algorithm*** that given an undirected graph $G = \langle V, E \rangle$ representing the current network decides whether the network is connected. A graph (the network) is ***connected*** if there is a path from any node to any other node in the graph. You may assume that G is given in the *adjacency list* format. Your algorithm must run in $O(n + m)$ time, where $n = |V|$ is the number of computers of the network, and $m = |E|$ is the number of connecting edges.

Input: Undirected graph G in adjacency list format
Output: True if connected, False otherwise

Pseudocode of IsConnected(G):

```
Starting node s
Initialize visited[n] to false
count = 0
Create a queue Q and enqueue s
visited[s] = true

while Q is not empty:
  u = Q.dequeue()
  count = count + 1
  for each neighbor v of u:
    if not visited[v]:
      visited[v] = true
      Q.enqueue(v)

if count == total_number_of_nodes:
  return True
else:
  return False
```

Starting from one node, the algorithm visits every node reachable via existing edges. If the total number of visited nodes equals n. Then every node is reachable from the start, which **confirming the connectivity**. It kind of look like **Bellman-Ford** but instead of calculating all the shortest path to reach from start node to every other node. This time we only check if start node can reach every node. Because **all the edges are undirected**, so if it can be reached, then it can reach to others as well.

- **Part C**

Given a graph $G = \langle V, E \rangle$, what is to be the runtime complexity of the ***depth-first search*** algorithm, as a function of the number of nodes $n = |V|$ and edges $m = |E|$, if the input graph is represented by an ***adjacency matrix*** instead of an *adjacency list*?

In **Depth-First Search** (DFS) using an **adjacency list**, the runtime is **O(n+m)** because we only look at existing edges. However, if an **adjacency matrix** is used. Then, to find the neighbours of a single node A for sample, the algorithm must scan the entire row of A which contain n columns. And so, since **DFS** visits **each of the n nodes**, it must **perform this n-column scan times**. With that, the total runtime is $\boldsymbol{O(n^2)}$**.**

- **Part D**

Consider the directed graph $G = \langle V, E \rangle$ represented by the following cost adjacency matrix:

$$A = \begin{bmatrix} . & 4 & . & . & . & 10 & . & . & . & . \\ . & . & . & 5 & 11 & . & 8 & 10 & . & . \\ . & . & . & . & 9 & . & . & . & . & . \\ 6 & . & 2 & . & . & 2 & . & . & . & . \\ . & . & . & . & 0 & . & . & 0 & . & 3 \\ . & . & . & 3 & . & . & . & . & . & . \\ . & 7 & . & . & . & 2 & . & . & . & . \\ . & . & 10 & . & . & . & . & . & . & . \\ . & . & . & . & . & . & . & 3 & . & . \\ . & . & . & 4 & . & . & . & . & . & . \end{bmatrix}$$

Assume that element a_{ij} positioned in row i and column j in matrix A stores the distance between node i and node j in graph G. Draw the graph and solve the single-source-shortest path problem by running the Dijkstra's algorithm on G, starting at node 1. What is the order in which nodes get removed from the associated priority queue? What is the resulting shortest-path tree?

Priority Queue (PQ)

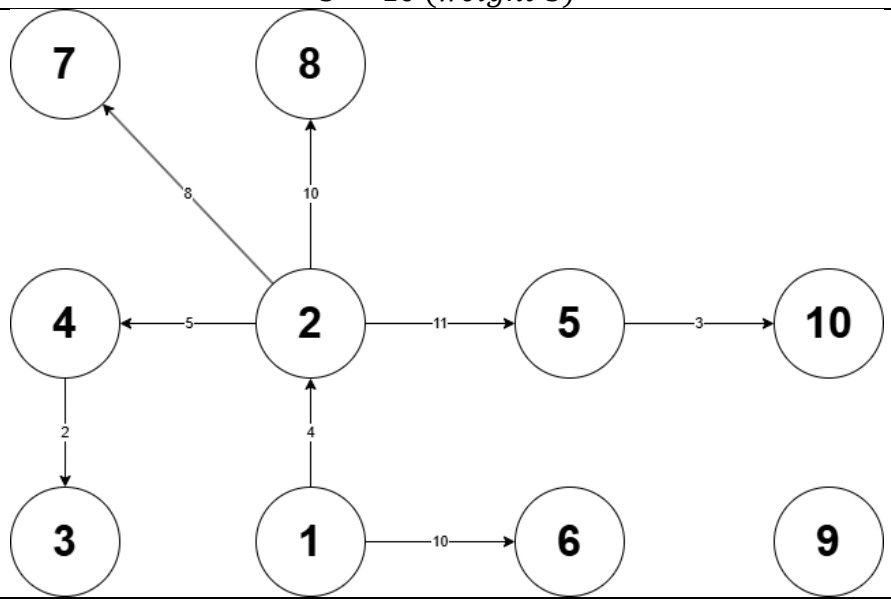
Current Node	Distance	Reached Path from node 1
1	0	1
2	4	1 2
3	11	1 2 4 3
4	9	1 2 4
5	15	1 2 5
6	10	1 6
7	12	1 2 7
8	14	1 2 8
9	∞	unreachable
10	18	1 2 5 10

Remove order: 1 2 4 6 3 7 8 5 10 9

Shortest Paths

$1 \rightarrow 2$ $(weight\ 4)$
$1 \rightarrow 6 (weight\ 10)$
$2 \rightarrow 4$ $(weight\ 5)$
$2 \rightarrow 5$ $(weight\ 11)$

$2 \rightarrow 7\ (weight\ 8)$
$2 \rightarrow 8\ (weight\ 10)$
$4 \rightarrow 3\ (weight\ 2)$
$5 \rightarrow 10\ (weight\ 3)$



- **Part E**

Let P be a shortest path from some vertex s to some other vertex t in a graph. If the weight of each edge in the graph is increased by one, then will P be still a shortest path from s to t? Explain your answer.

No, not every case though. Because the shortest paths depend **on the total sum of weights**. If **path A has 1 edge with weight 10** and **path B has 5 edges with weight 2 each**, they both originally total 10. For example, if adding 1 to every edge, then path A total would be $10 + 1 = 11$ and path B is now $10 + (5 \text{ x } 1) = 15$. So now, path A is now shorter than Path B. The algorithm now favors paths with **fewer edges.**

- **Part F**

You are given a graph G with exactly one cycle of negative weight (cost). You know the vertices that constitute the cycle. Design an algorithm that checks whether there exists a path from the source node to the destination with a negative total weight. You must explain the runtime complexity of your algorithm.

Input: Graph G, source s, destination t, set of vertices in the negative cycle
Output: True if a negative weight path exists, False otherwise

Pseudocode of CheckNegativeCycle(G, s, t, nodeList):

```
// check if startNode can reach negative node,
// and from negative node it can reach goal
negativeReachable = false
for each v in nodeList:
    if DFS_check(G, s, v):
        negativeReachable = true
        break
```

```
goalReachable = false
for each v in nodeList:
    if DFS_check(G, s, v):
        goalReachable = true
        break

if (negativeReachable AND goalReachable):
    return true

// ----------------------------------------------------
// if negativeNode is out of reach,
// then we use Bellman-Ford to find the shortest path from start to goal
dist[] = array of size n, initialize to infinity
dist[s] = 0 // set first node path cost to 0

for i from 1 to V - 1:
    for each edge (u,v) with weight w in E:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w

// check if the best path we found has negative total weight or not
if dist[t] < 0:
   return true
else
   return false
```

So, the **runtime complexity** is $\boldsymbol{O(n \cdot m)}$**,** where n is the number of vertices and m is the number of edges. Even though, the reachability (DFS, we can also use BFS) take total $\boldsymbol{O(n + m)}$ time. But **Bellman-Ford** iterates through all m edges n – 1 time, resulting in $0(n \cdot m)$. In the end, $0(n \cdot m)$ grow faster than $O(n + m)$ so $0(n \cdot m)$ is the final runtime.